

TP3 Machine à états (FSM)

Question 1

Dans un fichier .vhd, créez un module `Counter_unit` à partir du compteur du TP1. Le module prendra en entrée un signal d'horloge et de `resetn`, et donnera en sortie le signal `end_counter`. Utilisez un paramètre `generic()` pour définir le nombre de coup d'horloge à compter.

Entité

Configuration

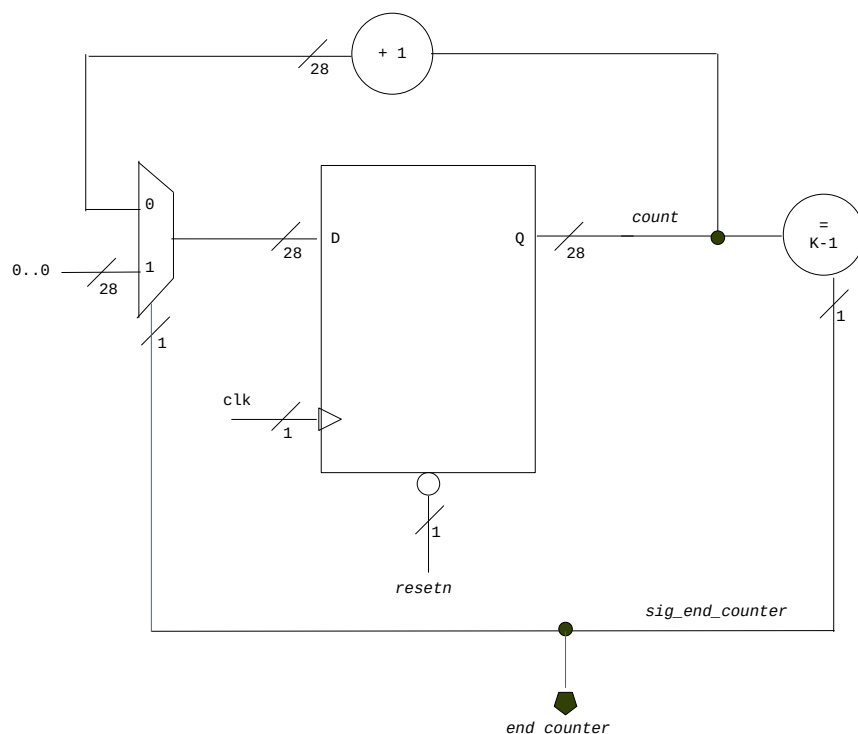
Constante *generic* `K`, de type `positive` : période compteur exprimée en périodes horloge.

Entrées / sorties

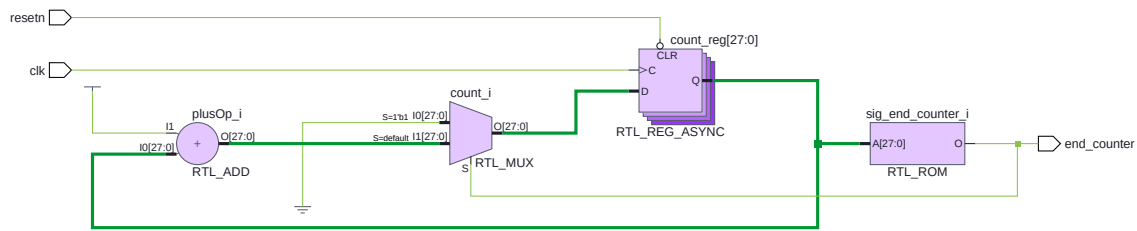
Port	Mode	Signal
<code>clk</code>	in	Horloge
<code>resetn</code>	in	Reset (asynchrone)
<code>end_counter</code>	out	Fin de cycle compteur (synchrone)

Toutes les entrées/sorties sont de type `std_logic`.

RTL



Schematic



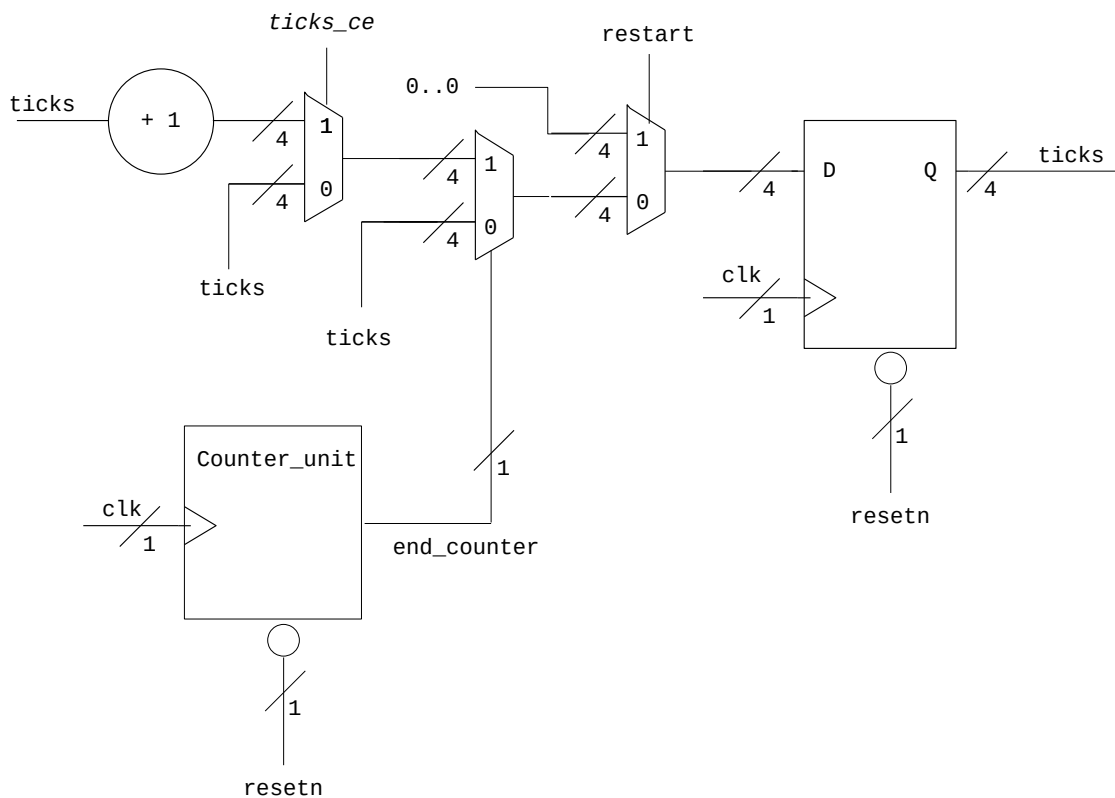
VHDL

Source VHDL : src/counter_unit.vhd

Question 2

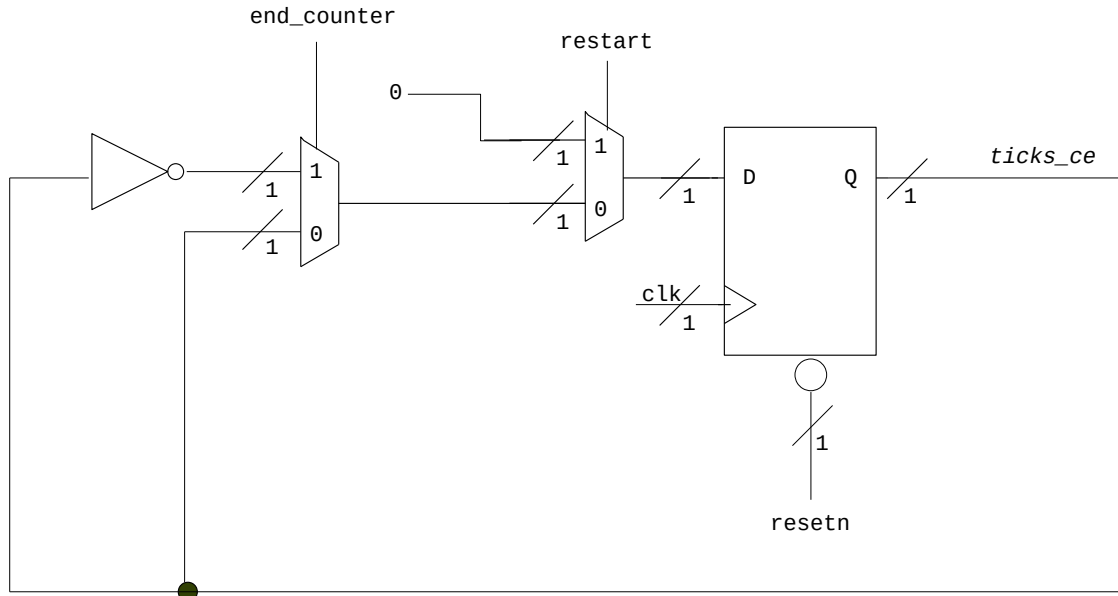
En schéma RTL, créez un compteur du signal `end_counter`. Ce compteur doit permettre de déterminer le nombre de cycles allumé/éteint qui ont été effectués par la LED. Le compteur doit pouvoir être remis à 0, maintenir sa valeur actuelle ou s'incrémenter.

Compteur



Signal ticks_ce

Le signal `ticks_ce` détermine si le signal d'entrée `end_counter` doit effectivement incrémenter le compteur : souhaitant compter l'équivalent des périodes ON/OFF, `ticks_ce` sera HIGH eulement un évènement `end_counter` sur deux.



Question 3

Ecrivez un code VHDL décrivant ce compteur de cycle, vous utiliserez le module `Counter_unit`.

Le compteur de cycle est défini par l'entité `Fsm_ticks`, dans le fichier `src/fsm_ticks.vhd`.

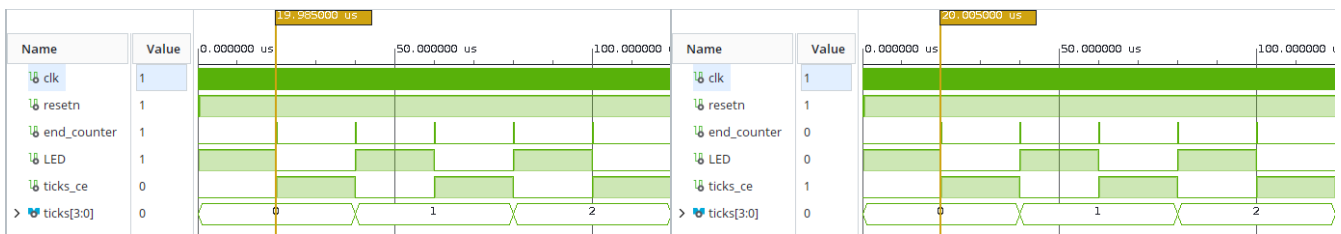
Question 4

Tester votre architecture avec un testbench.

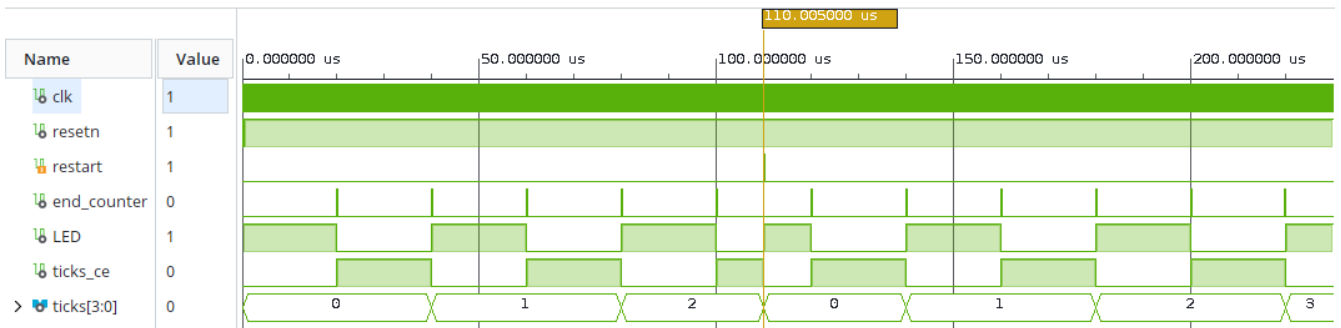
Testbench pour Le comportement de l'entité `Fsm_ticks`, est testé dans le `testbench` `src/tb_fsm_ticks.vhd`.

Tests comportement

	<i>t</i> (ns)	LED	end_counter	ticks_ce	ticks
<i>t</i> 0	1	0	0	0	
19985	1	1	0	0	
19995	0	0	1	0	
39995	1	0	0	1	

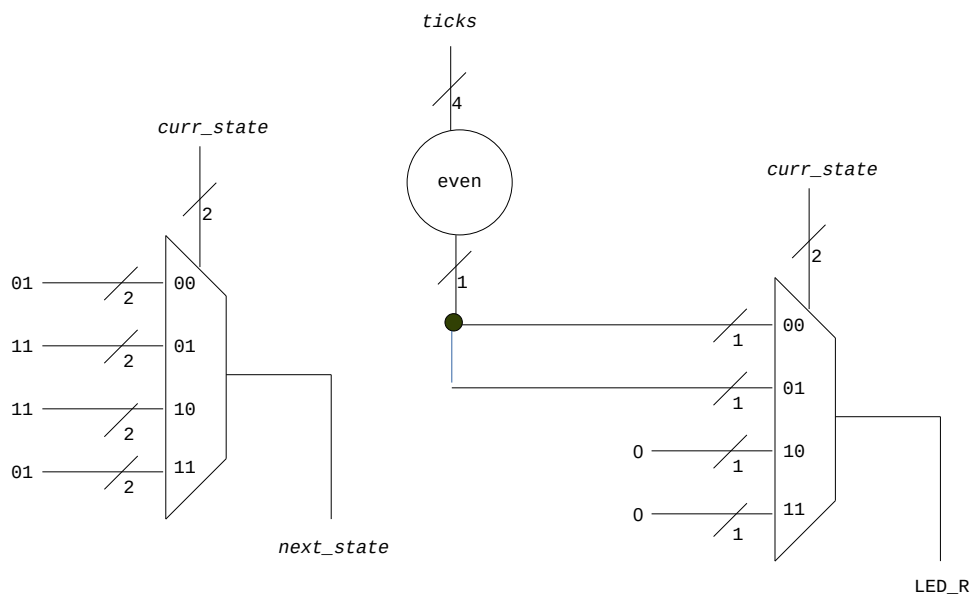


Test comportement sur restart



Question 5

Créez en RTL une machine à états (FSM) permettant de faire clignoter une LED RGB en rouge puis bleu et enfin en vert avant de recommencer le cycle (rouge, bleu, vert, ...). Dans chaque état la LED devra clignoter 3 fois. De plus, si le bouton restart est appuyé, on retourne dans l'état initial quel que soit l'état dans lequel on se situe. L'état initial est l'état dans lequel on se situe au démarrage, on passe à l'état rouge après 3 clignotements de la LED en blanc (rouge, vert et bleu actifs en même temps)



Le schéma ci-dessus rappelle les principaux éléments de la machine à états :

- Comment calculer l'état suivant à partir de l'état courant.
- Comment déterminer le niveau logique d'une LED à partir de l'état courant et du nombre d'évènements ticks.

Note : on a conservé un compteur sur 4 bits, permettant de plus riches simulations, bien que inutilement large pour cette machine à états (3 bits suffiraient).

Question 6

Listez les signaux d'entrée, de sortie et les signaux internes de votre architecture.

Ports d'entrées sorties

Port	Mode	Signal
clk	in	Horloge
resetn	in	Reset (asynchrone)
restart	in	Redémarrage (synchrone)
LED_R	out	LED channel Red
LED_B	out	LED channel Blue
LED_G	out	LED channel Green

Signaux internes

- Signaux de connexion entre les composants Counter_unit (end_counter) et Fsm_ticks (ticks)
- Signaux internes à la machine à états : curr_state et next_state

Question 7

Ajoutez à votre code VHDL les éléments que vous venez de créer.

Voir src/fsm.vd.

Question 8

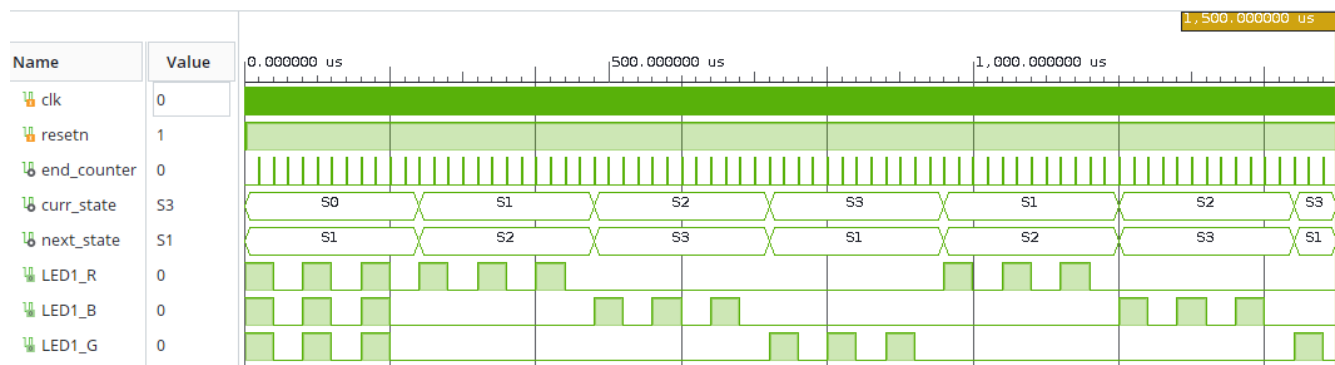
Ecrivez un testbench pour tester votre architecture. Vérifiez à la simulation que vous obtenez le résultat attendu.

Voir src/tb_fms.vhd.

Tests comportement

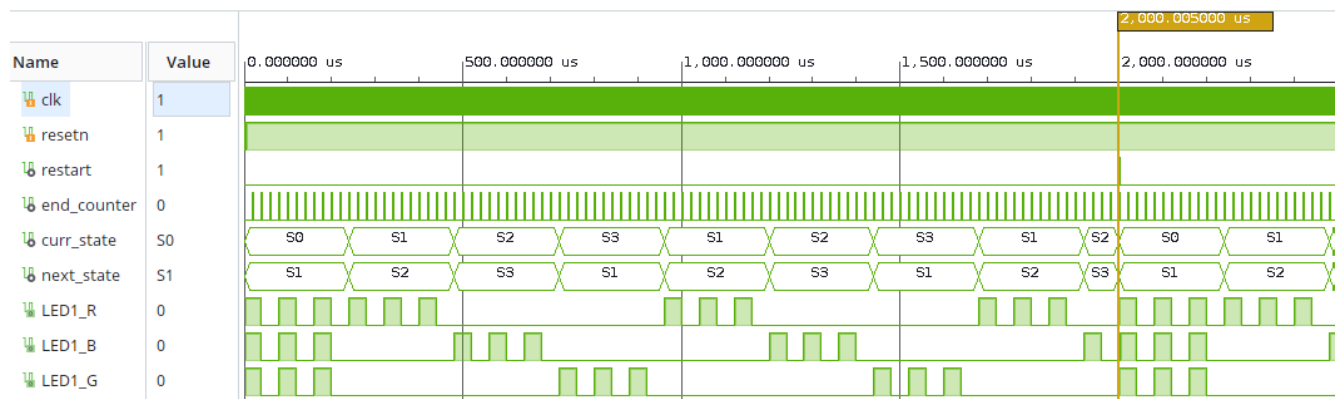
Vérification du comportement normal de la machine à états :

- État initial S0, LEDs R, B et G allumées
- État S1, LED R allumée, LEDs B et G éteintes
- État S2, LED B allumée, LEDs R et G éteintes
- État S3, LED G allumée, LEDs R et B éteintes
- Retour à l'état S1



Tests comportement sur restart

On vérifie que le signal restart ré-initialise la séquence ci-dessus.



Question 9

Exécutez la synthèse et relevez les ressources utilisées (y compris la FSM). Sur la schématique, identifiez où se situe votre compteur de cycle.

Le rapport est dans rpt/utilization_place_design.txt.

TBD : identification du compteur de cycle.

Question 10

Modifiez le fichier de contraintes pour connecter vos entrées / sorties du système avec les broches de la carte. Réglez l'horloge pour que sa fréquence soit à 100MHz.

Voir src/Cora-Z7-10-Master.xdc.

Question 11

Lancez l'implémentation puis étudiez le rapport de timing (vérifiez les violations de set up et de hold et identifiez le chemin critique).

Le rapport est dans rpt/timing_summary.txt.

Question 12

Générez le bitstream pour vérifier le système sur carte.

Note : L'entité finalement programmée est top_fsm, définie dans src/top_fsm.vhd., afin de connecter la machine à états aux boutons et LEDs de la carte.